

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: Система управления и мониторинга роботов

Студенты гр. 3388

Беннер В.А.
Лутфулин Д.А.
Скрябина С.М.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты:

Беннер В.А.

Лутфулин Д.А.

Скрябина С.М.

Группа 3388

Тема работы: Разработка системы управления и мониторинга роботов (RMMS - Robot Management and Monitoring System).

Исходные данные:

Нужно сделать приложение, решающее задачу хранения информации о Н группах роботов. Храним как метаданные самой группы, так и метаданные конкретных роботов, их координаты, траектории, бинарные данные (кадры с камеры), храним текущие задания для группы, логи. Визуализируем положение на карте.

Содержание пояснительной записки:

Содержание, Введение, Качественные требования к решению, Сценарии использования (Use Cases), Макет UI, Нереляционная модель данных (MongoDB), Реляционная модель данных (SQL), Сравнение моделей, Вывод

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 24.05.2026

Дата защиты реферата: 24.05.2026

Студенты

Беннер В.А.
Лутфулин Д.А.
Скрябина С.М.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

В работе выполнено проектирование, расчет занимаемой памяти и детальное сопоставление реляционного и документоориентированного подходов к построению хранилища системы управления и мониторинга групп роботов. Приложение разработано с учетом критериев масштабируемости и эффективного хранения телеметрии и тяжелых бинарных данных. В результате работы построены обе архитектуры БД, проведено их сравнение, выявившее линейный превосходство NoSQL-структуры в данной задаче. На основе спроектированных моделей разработано веб-приложение, реализующее основные сценарии использования.

SUMMARY

This work presents the design, memory calculation, and detailed analysis of relational and document-oriented approaches to building database for control systems and robot groups. The application was developed with scalability, including telemetry storage and binary data validation. The development of both database designs, coupled with their comparison, revealed the superiority of the NoSQL structure for this task. Based on the proposed models, a web application was developed that implements the main use cases.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1. КАЧЕСТВЕННЫЕ ТРЕБОВАНИЯ К РЕШЕНИЮ.....	8
2. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ (USE CASES)	9
3. МАКЕТ UI	14
4. НЕРЕЛЯЦИОННАЯ МОДЕЛЬ ДАННЫХ (MONGODB)	19
5. РЕЛЯЦИОННАЯ МОДЕЛЬ ДАННЫХ (SQL).....	25
6. СРАВНЕНИЕ МОДЕЛЕЙ	30
РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ	36
ЗАКЛЮЧЕНИЕ.....	42
ПРИЛОЖЕНИЕ А.....	43
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	44

ВВЕДЕНИЕ

Настоящая работа посвящена проектированию и сравнительному анализу архитектуры баз данных для систем управления и мониторинга флота автономных роботов (RMMS — Robot Management and Monitoring System). Подобные системы активно применяются в современной промышленной автоматизации, на роботизированных складах и в инфраструктуре умных городов.

Основная проблема при разработке RMMS заключается в том, что база данных должна одновременно справляться с тремя абсолютно разными по своей природе типами нагрузок:

1. **Транзакционная нагрузка (Управление):** Хранение настроек роботов, распределение их по логическим группам и создание комплексных задач (миссий). Эти данные требуют абсолютной надежности, строгой целостности связей и поддержки транзакций. Ошибки или потеря записей здесь недопустимы, так как это приведет к потере контроля над устройствами.
2. **Интенсивные временные ряды (Телеметрия):** Непрерывный поток координатных и технических точек движения от каждого активного робота (координаты x и y , скорость, направление, остаток заряда батареи), присылаемый с определенной частотой. При росте количества роботов нагрузка на запись растет лавинообразно. База данных должна обладать колоссальной пропускной способностью, чтобы оператор видел перемещения флота на карте в реальном времени без задержек.
3. **Тяжелые бинарные данные (BLOB):** Фотоснимки высокого разрешения, которые роботы отправляют на сервер в момент обнаружения препятствий или аварийных ситуаций. Каждое такое изображение весит около 1 МБ и должно быть жестко привязано к конкретному логу события, не замедляя при этом поиск по обычным текстовым или числовым полям. Попытка реализовать такое гибридное хранилище на базе одной СУБД часто упирается в архитектурные ограничения конкретных систем. В данной

работе проводится проектирование, расчет занимаемой памяти и детальное сопоставление двух принципиально разных подходов к построению хранилища RMMS:

1. **Реляционный подход (SQL):** Структурированное хранение данных, жестко разделенных по таблицам (роботы, миссии, события) с гарантией ссылочной целостности, но требующее ресурсоемких операций объединения (JOIN) для сборки полной информации.
2. **Документориентированный подход (NoSQL / MongoDB):** Хранение данных в виде гибких JSON-документов с вложенными структурами (когда маршруты и списки роботов лежат прямо внутри документа задачи), что позволяет читать данные атомарно без склейки таблиц.

Цель работы — определить наиболее эффективную модель базы данных для системы RMMS на основе математического расчета объема занимаемой памяти, анализа избыточности денормализованных данных и оценки вычислительной сложности ключевых поисковых запросов и спроектировать её.

1. КАЧЕСТВЕННЫЕ ТРЕБОВАНИЯ К РЕШЕНИЮ

Информационная система RMMS должна удовлетворять следующим качественным критериям:

1. **Масштабируемость (Scalability):** Способность системы линейно увеличивать пропускную способность записи телеметрии при росте размера флота до сотен активных роботов.
2. **Низкая задержка на чтение (Low Read Latency):** Отображение позиций всех роботов на карте в реальном времени требует от БД извлечения последних координат флота за время, не превышающее 50–100 мс.
3. **Эффективное хранение бинарных данных (Blob Storage Efficiency):** Фотоснимки с камер роботов должны быть жестко ассоциированы с метаданными логов событий, не вызывая деградации производительности индексов при поиске по текстовым и числовым полям.
4. **Обеспечение целостности связей:** Предотвращение удаления групп или миссий, в которых в данный момент задействованы активные аппаратные единицы.

2. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ (USE CASES)

Основным действующим лицом во всех сценариях является Пользователь (Оператор) системы.

2.1. Управление роботами и группами (Dashboard)

УС-01: Просмотр и фильтрация списка роботов

Цель: Получить актуальную информацию о статусе и местоположении роботов, используя гибкие критерии поиска.

Предусловие: Оператор находится на вкладке Dashboard.

Основной сценарий:

- Система отображает две основные панели: "Роботы" и "Группы".
- На панели "Роботы" система выводит список всех роботов, отсортированных по времени последнего обновления (сверху новые). Для каждого робота отображается минимум: Имя, Статус, Группа, Уровень заряда, время последнего обновления.
- Над списком роботов система отображает панель расширенного поиска и фильтрации, содержащую отдельные поля для каждого атрибута: Поле ввода (поиск по подстроке, регистронезависимый), выпадающие списки "Статус", "Группа", "Тип робота", "Тип миссии", два поля "Дата:Время" для выбора временного промежутка, два поля для ввода числовых значений "Батарея".
- Оператор выбирает критерии фильтрации (например, группа "Alpha", статус "Idle").
- Система в реальном времени фильтрует список по конъюнкции условий.
- Оператор нажимает на имя любого робота в списке для перехода к карточке робота (УС-02).

Альтернативные сценарии:

- Список пуст: Если в системе нет роботов, система отображает информационное сообщение и кнопку "Добавить робота".
- Нет результатов: Отображается пустой список.
- Сброс фильтров: Оператор нажимает кнопку "Сбросить", система возвращает полный список.

УС-02: Просмотр детальной информации и управление роботом

Цель: Просмотреть подробную статистику конкретного робота, отредактировать его параметры или удалить его.

Предусловие: Оператор выбрал робота из списка (УС-01, п.6).

Основной сценарий:

- Система открывает страницу детальной информации о роботе.
- На странице отображается таблица, содержащая всю информацию о роботе. Под таблицей расположены ссылки на последние сделанные фото с поиском по названию, типу сортировки, временному промежутку создания фото и миссии, в рамках которой было сделано фото.
- Оператор нажимает кнопку "Редактировать". Система открывает форму редактирования (Имя, Группа, комментарий). Оператор изменяет данные и нажимает "Сохранить". Система валидирует данные, сохраняет изменения и обновляет страницу.
- Оператор нажимает кнопку "Удалить". Система запрашивает подтверждение. После подтверждения робот удаляется из БД, а оператор перенаправляется на Dashboard.

Альтернативные сценарии:

- Отмена редактирования: Форма закрывается без сохранения изменений.
- Ошибка валидации: Поля подсвечиваются красным, форма остается открытой с выводом сообщения об ошибке.

УС-03: Добавление новой сущности (Робот, Группа)

Цель: Добавить нового робота или новую группу роботов.

Предусловие: Оператор находится на вкладке Dashboard.

Основной сценарий (Добавление робота):

- На панели "Роботы" Оператор нажимает кнопку "Create New Robot".
- Система открывает модальное окно с формой создания робота (Имя, Модель, Группа).
- Оператор заполняет поля и нажимает "Создать". Робот регистрируется в БД, открывается страница его деталей.

Основной сценарий (Добавление группы):

- На панели "Группы" Оператор нажимает кнопку "Create New Group".
- В форме вводится название группы. Оператор нажимает "Создать".
- Система проверяет уникальность названия, создает группу и обновляет список групп на Dashboard.

2.2. Планирование заданий (Task Planner)

УС-04: Управление миссиями (заданиями)

Цель: Создать новую миссию для группы роботов, отслеживать её выполнение, отменять или просматривать результаты завершенных миссий.

Предусловие: Оператор перешел на вкладку Task Planner.

Основной сценарий:

- Система отображает интерфейс планировщика, разделенный на "Карту/Редактор (Map/Editor)" для построения маршрута и "Список миссий" (Группа, Тип, Время запуска, Текущий статус).
- Оператор нажимает кнопку "Add Task". Система открывает панель создания миссии: выбор Группы, выбор Типа миссии, возможность указать координаты/маршрут на карте.
- Оператор заполняет данные, взаимодействуя с картой, и нажимает "Launch". Система валидирует доступность группы и координаты рабочей зоны, переводит миссию в список "Активные".

- Оператор может выбрать миссию в списке и нажать "Cancel" для её досрочной отмены.

Альтернативные сценарии:

- Координаты вне зоны: Выводится ошибка "Invalid coordinates", миссия не сохраняется.

2.2. Мониторинг (Map)

UC-05: Мониторинг на карте

Цель: Визуально отслеживать текущее местоположение и статус всех активных роботов в реальном времени.

Предусловие: Оператор перешел на вкладку Map.

Основной сценарий:

- Система отображает полноэкранную карту рабочей зоны.
- На карту наносятся маркеры всех роботов со статусом, отличным от "Offline". Цвет маркера отражает статус (зеленый — Idle, синий — Moving, желтый — Charging).
- При нажатии на маркер робот подсвечивается. Справа от карты видна иконка робота, клик по которой перенаправляет на панель робота (UC-02).
- Позиции роботов обновляются в реальном времени.

UC-06: Просмотр и экспорт аналитики

Цель: Отслеживать события системы в реальном времени, фильтровать их по различным критериям и экспортировать отчет.

Предусловие: Оператор перешел на вкладку Analytics.

Основной сценарий:

- Система отображает ленту событий (Critical Events), панель фильтрации, быстрые фильтры времени (24h, 7h, 30m, ALL), поле полнотекстового поиска и блок экспорта.

- Оператор применяет фильтры. Система обновляет ленту событий.
- Оператор нажимает кнопку "Start Export". Система формирует отчет о деятельности флота (PDF/JSON) и предлагает сохранить его на локальный диск.

УС-07: Управление системными данными (Импорт/Экспорт)

Цель: Выполнить импорт пользовательских метрик или экспорт полного состояния системы для офлайн-анализа.

Предусловие: Оператор перешел на вкладку System Settings (заголовок версии "Robot Fleet v2.4.1").

Основной сценарий:

- В разделе "Data Management" оператор нажимает "Upload Own Data", выбирает локальный файл с метриками и подтверждает загрузку. Система обрабатывает файл и выводит уведомление.
- Оператор нажимает "Download Data". Система формирует архив конфигурации флота и мастер-логов и скачивает его на ПК оператора.

3. MAKET UI

Макет пользовательского интерфейса описывает взаимное расположение компонентов Dashboard, Карты, Ленты событий и Панели управления, спроектированных в соответствии со сценариями UC-01 – UC-07.

1. Экран Dashboard

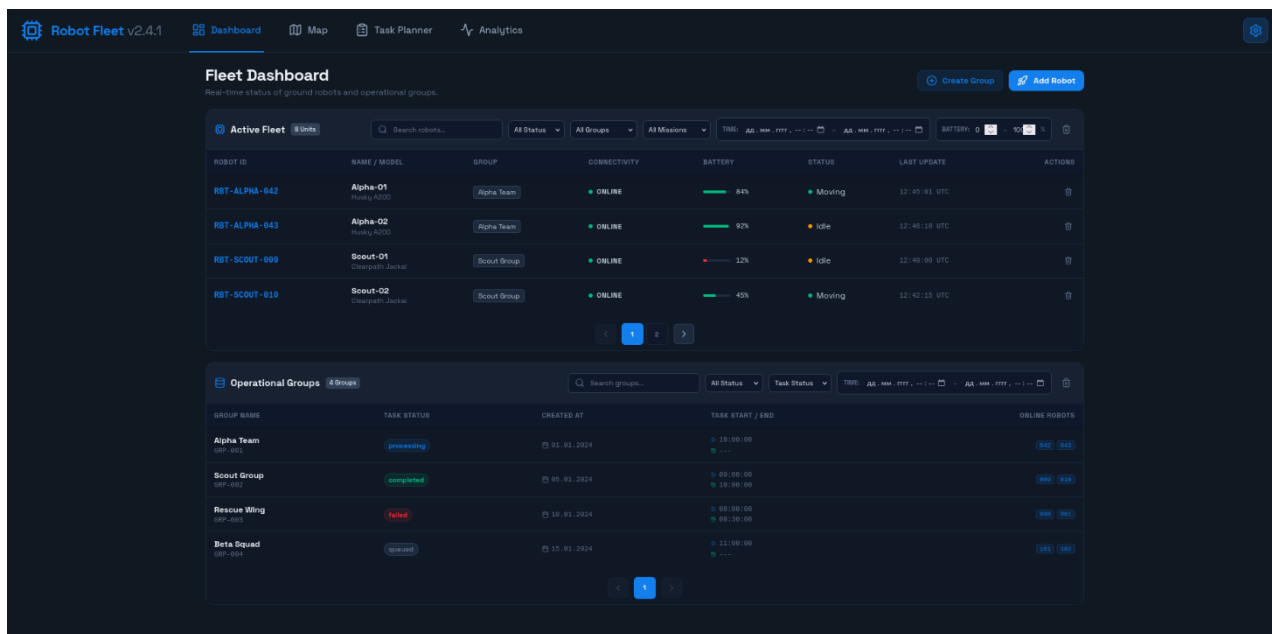


Рис.1 — экран Dashboard

2. Экран Analytics

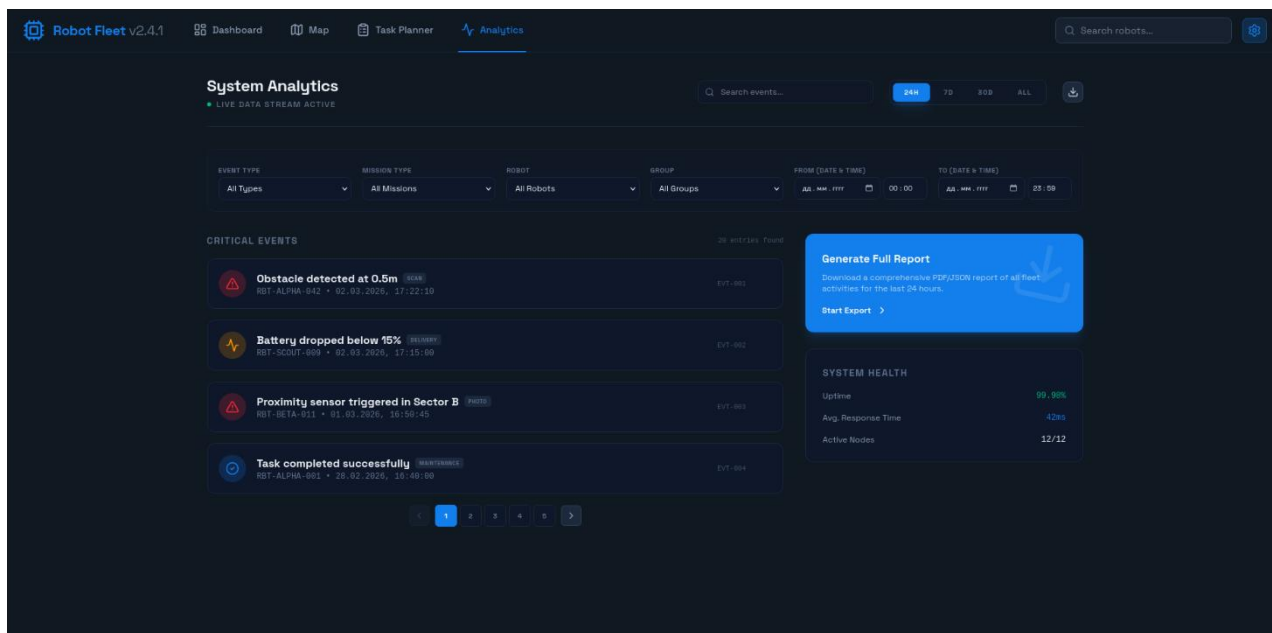


Рис.2 — экран Analytics

3. Экран Task Planner

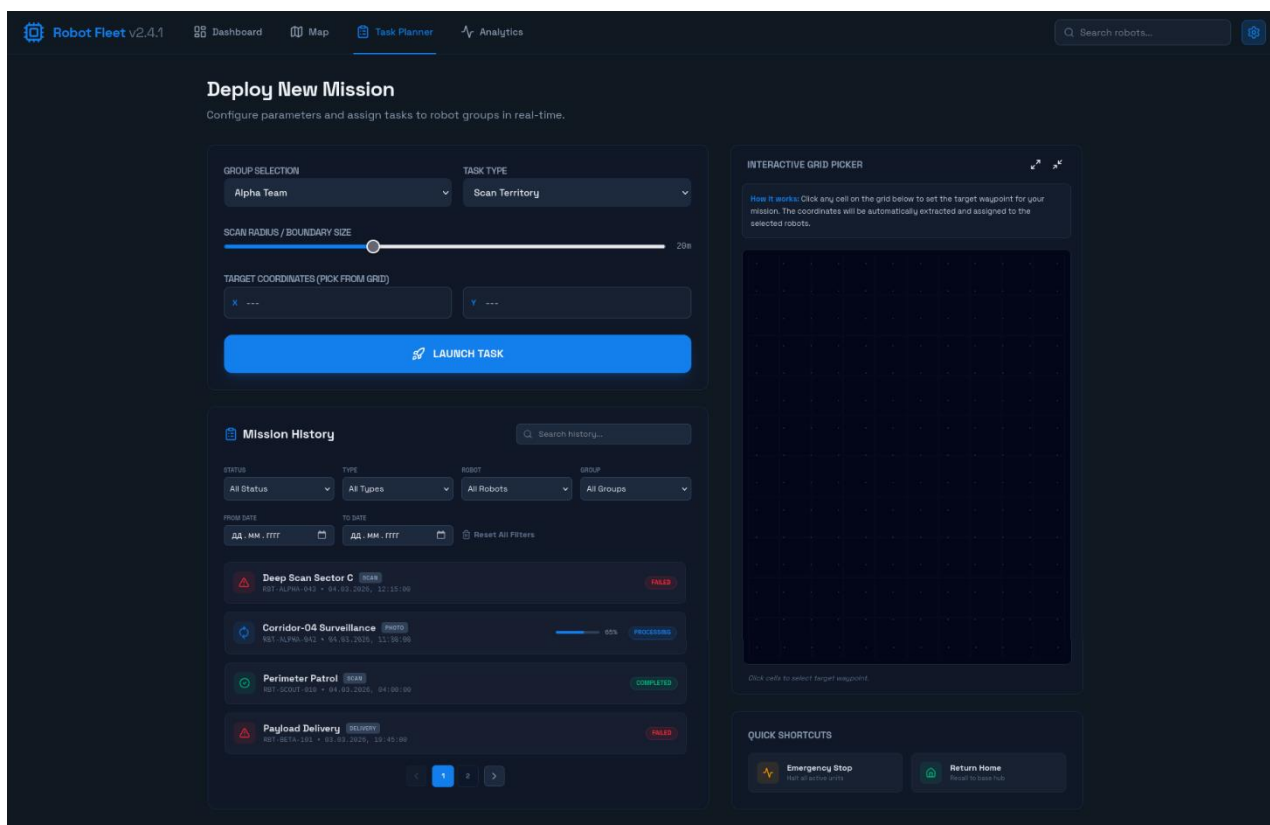


Рис.3 — экран Task Planner

4. Экран Map

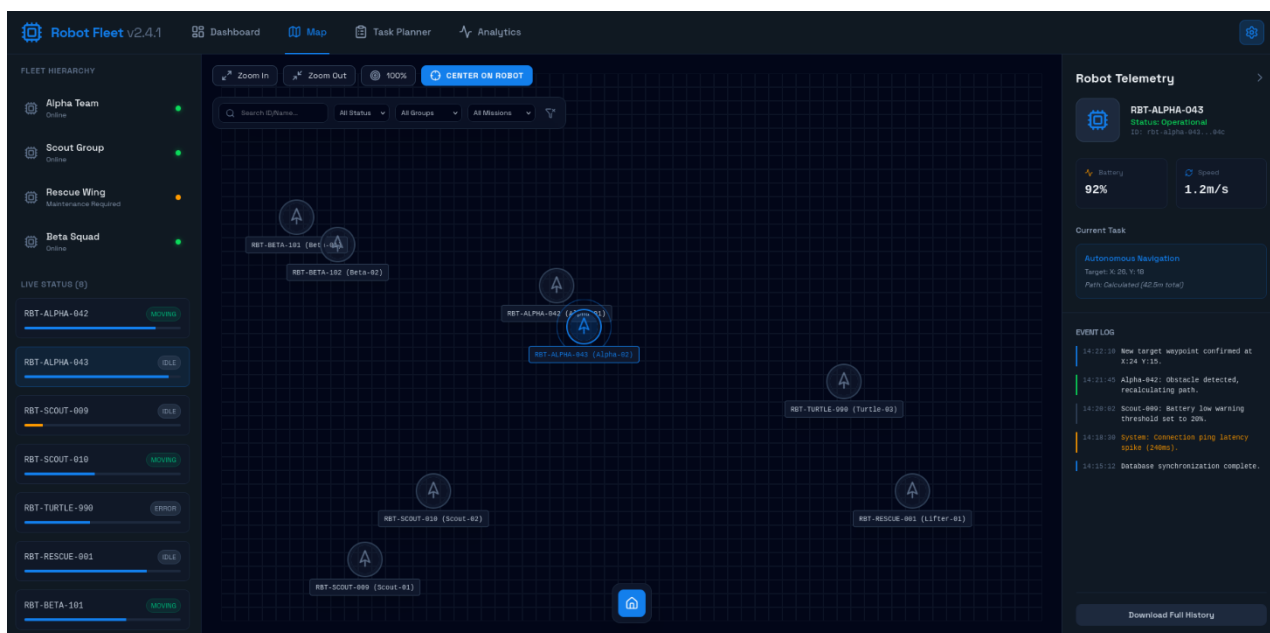


Рис.4 — экран Map

5. Экран Robot board

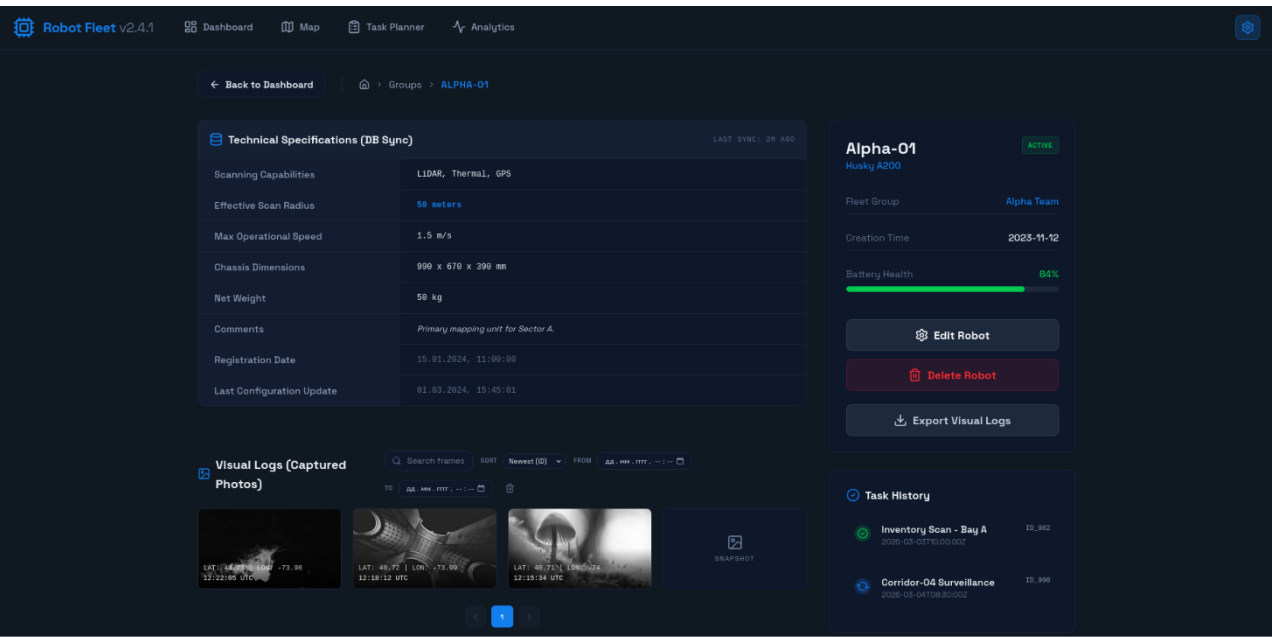


Рис.5 — экран Robot board

6. Экран Group board

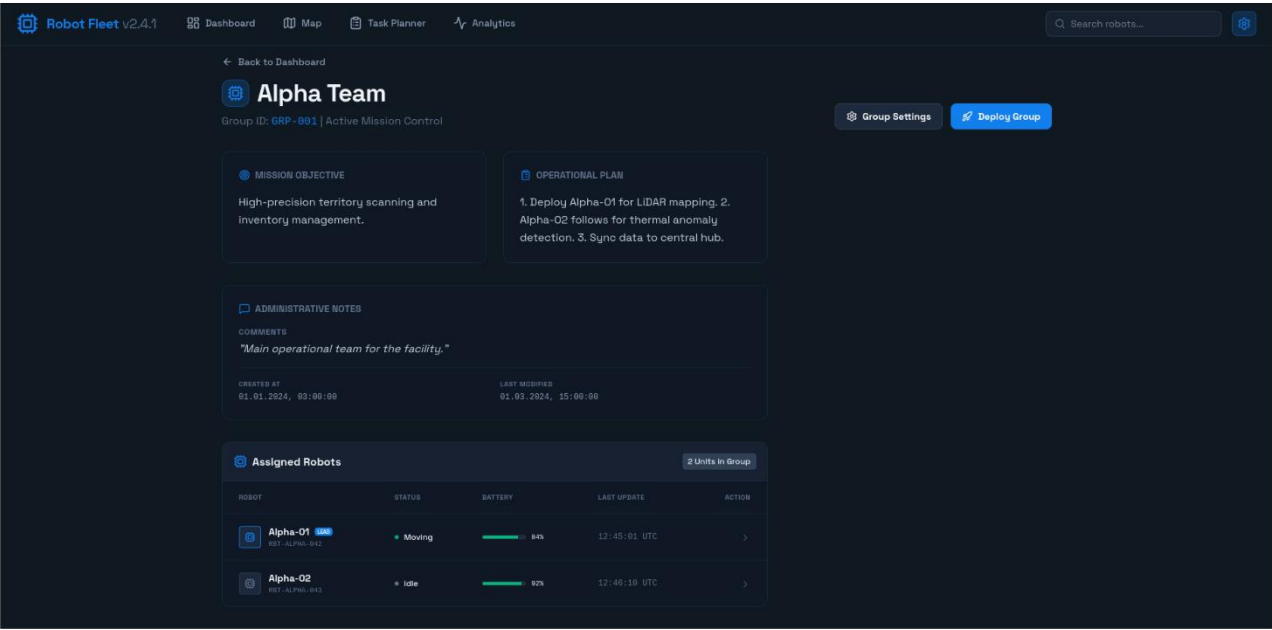


Рис.6 — экран Group board

7. Экран System Settings

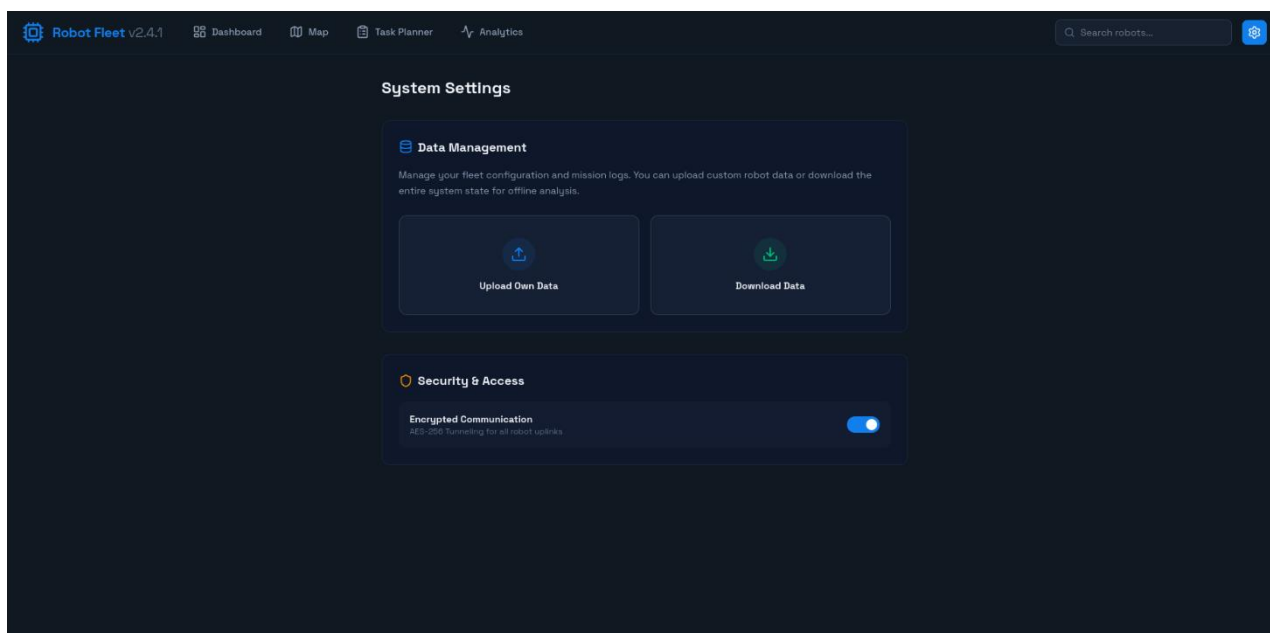


Рис.7— экран System Settings

8. Всплывающие окна для создания группы и роботами

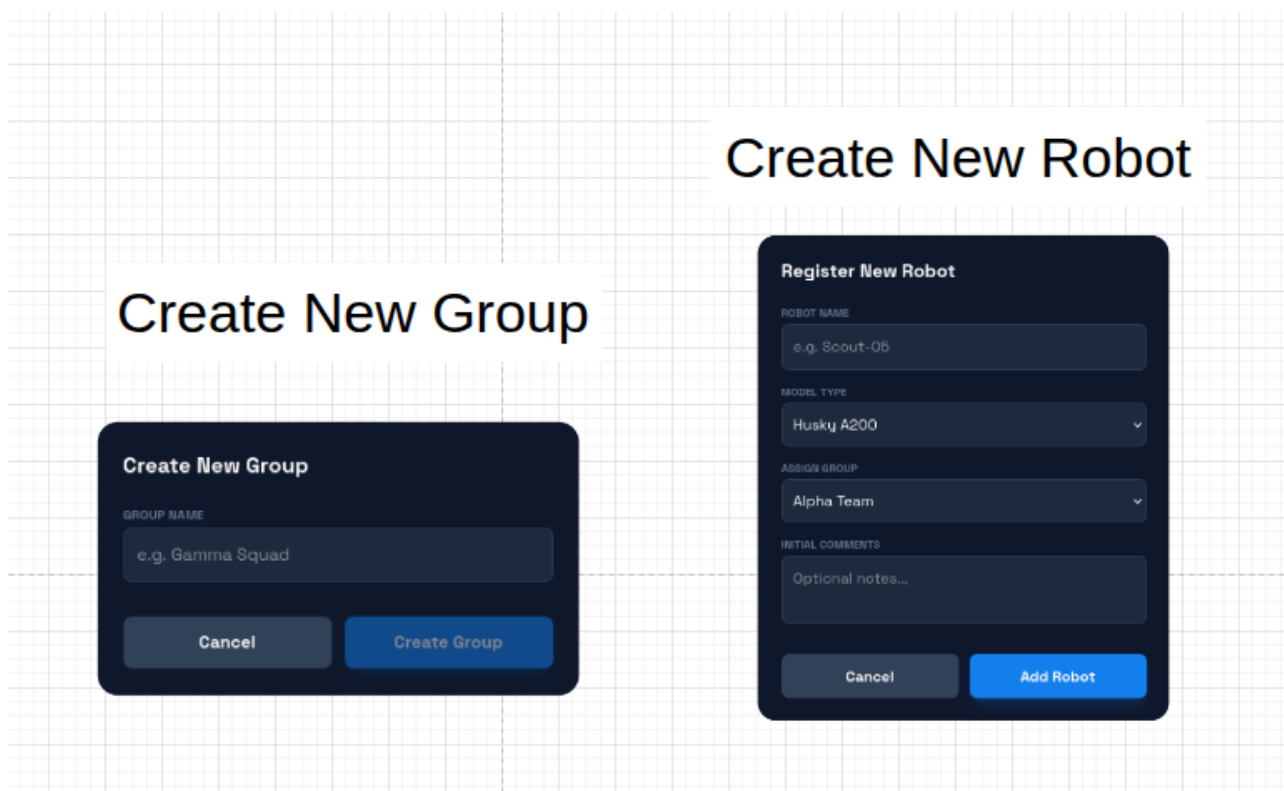


Рис.8— окна Create New Group и Create New Robot

9. Всплывающие окна для редактирования параметров группы и роботов

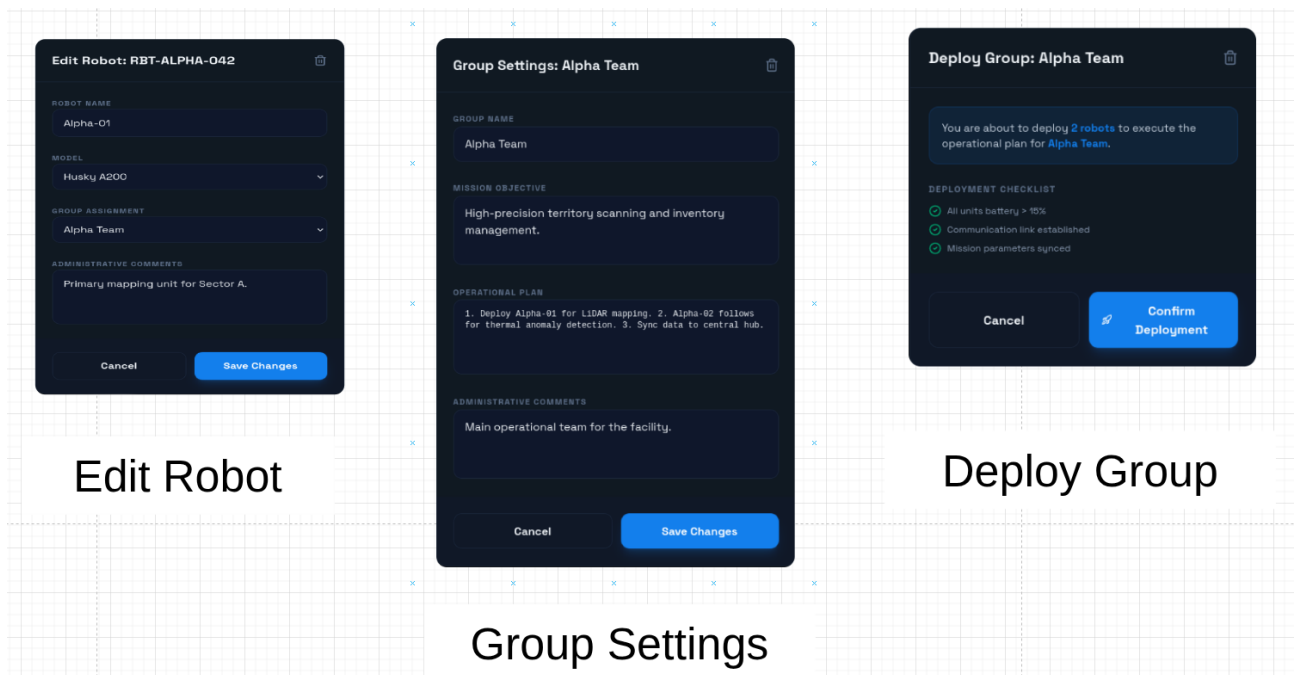


Рис.9— окна Edit Robot, Group Settings и Deploy Group

10. Всплывающие окна работы с задачами

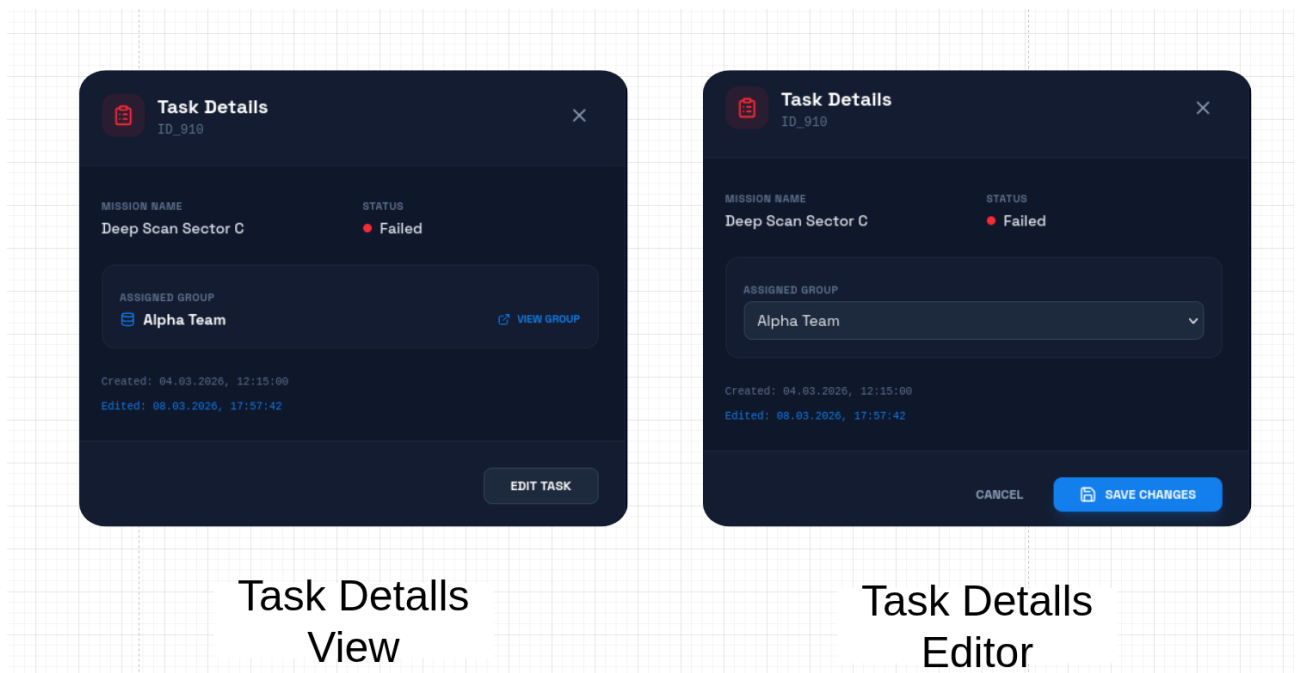


Рис.10— окна Task Details View и Task Details Editor

4. НЕРЕЛЯЦИОННАЯ МОДЕЛЬ ДАННЫХ (MONGODB)

Документоориентированная модель спроектирована с использованием принципа частичной денормализации для устранения необходимости ресурсоемких операций объединения (JOIN) при выводе списков роботов и их текущих задач.

4.1. Графическое представление модели

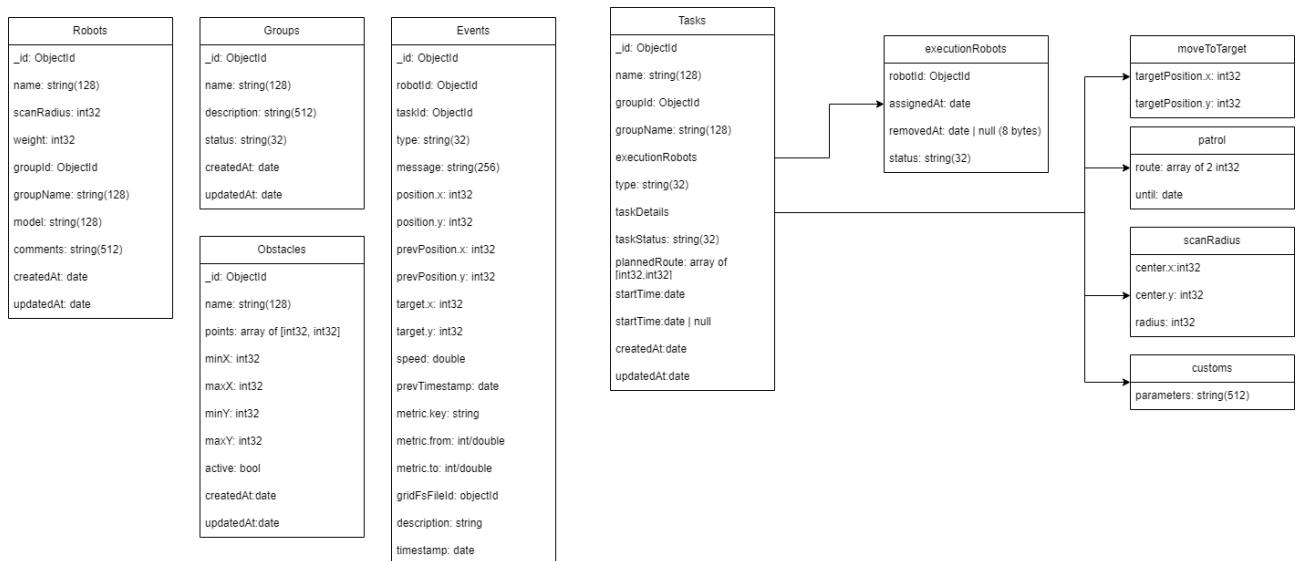


Рис.11 - Графическое представление модели

4.1. JSON-схемы валидации коллекций

Коллекция Robots

```
"bsonType": "object",
"required": ["name", "groupId", "model", "createdAt", "updatedAt"],
"properties":
  "name": "bsonType": "string" ,
  "scanRadius": "bsonType": "int" ,
  "weight": "bsonType": "int" ,
  "groupId": "bsonType": "objectId" ,
  "groupName": "bsonType": "string" ,
  "model": "bsonType": "string" ,
  "comments": "bsonType": "string" ,
  "createdAt": "bsonType": "date" ,
```

```
"updatedAt": "bsonType": "date"
```

Коллекция Groups

```
"bsonType": "object",  
"required": ["name", "status", "createdAt", "updatedAt"],  
"properties":  
  "name": "bsonType": "string" ,  
  "description": "bsonType": "string" ,  
  "status":  
    "bsonType": "string",  
    "enum": ["active", "inactive", "paused", "error"]  
  ,  
  "createdAt": "bsonType": "date" ,  
  "updatedAt": "bsonType": "date"
```

Коллекция Events

```
"bsonType": "object",  
"required": ["robotId", "type", "timestamp"],  
"properties":  
  "robotId": "bsonType": "objectId" ,  
  "taskId": "bsonType": ["objectId", "null"] ,  
  "type":  
    "bsonType": "string",  
    "enum": ["battery_low", "battery_critical", "battery_delta", "task_start", "task_complete", "task_failed",  
"error", "warning", "info", "track_point", "status_change", "metric_change", "visual_capture"]  
  ,  
  "message": "bsonType": "string" ,  
  "position":  
    "bsonType": "object",  
    "properties":  
      "x": "bsonType": "int" ,  
      "y": "bsonType": "int"  
    ,  
  ,
```

```

"prevPosition":
  "bsonType": "object",
  "properties":
    "x": "bsonType": "int" ,
    "y": "bsonType": "int"

,

"target":
  "bsonType": "object",
  "properties":
    "x": "bsonType": "int" ,
    "y": "bsonType": "int"

,

"speed": "bsonType": "double" ,
"prevTimestamp": "bsonType": "date" ,
"metric":
  "bsonType": "object",
  "properties":
    "key": "bsonType": "string" ,
    "from": "bsonType": ["double", "int", "null"] ,
    "to": "bsonType": ["double", "int", "null"]

,

"gridFsFileId": "bsonType": ["objectId", "null"] ,
"description": "bsonType": "string" ,
"timestamp": "bsonType": "date"

```

Коллекция Tasks

```

"bsonType": "object",
"required": ["groupId", "type", "taskStatus", "createdAt"],
"properties":
  "name": "bsonType": "string" ,
  "groupId": "bsonType": "objectId" ,
  "groupName": "bsonType": "string" ,
  "executionRobots":

```

```

    "bsonType": "array",
    "items":
      "bsonType": "object",
      "required": ["robotId", "assignedAt", "status"],
      "properties":
        "robotId": "bsonType": "objectId" ,
        "assignedAt": "bsonType": "date" ,
        "removedAt": "bsonType": ["date", "null"] ,
        "status":
          "bsonType": "string",
          "enum": ["assigned", "working", "completed", "removed"]

,
"type":
  "bsonType": "string",
  "enum": ["moveToTarget", "patrol", "scanRadius", "custom"]
,
"taskDetails": "bsonType": "object" ,
"taskStatus":
  "bsonType": "string",
  "enum": ["active", "paused", "completed", "cancelled", "failed"]
,
"plannedRoute":
  "bsonType": "object",
  "properties":
    "points":
      "bsonType": "array",
      "items":
        "bsonType": "object",
        "required": ["x", "y"],
        "properties":
          "x": "bsonType": "int" ,
          "y": "bsonType": "int"

```

```
,
"startTime": "bsonType": "date",
"endTime": "bsonType": ["date", "null"],
"createdAt": "bsonType": "date",
"updatedAt": "bsonType": "date"
```

Коллекция Obstacles

```
"bsonType": "object",
"required": ["points", "minX", "maxX", "minY", "maxY", "createdAt"],
"properties":
  "points":
    "bsonType": "array",
    "items":
      "bsonType": "array",
      "items": "bsonType": "int"

  ,
  "minX": "bsonType": "int",
  "maxX": "bsonType": "int",
  "minY": "bsonType": "int",
  "maxY": "bsonType": "int",
  "name": "bsonType": "string",
  "active": "bsonType": "bool",
  "createdAt": "bsonType": "date",
  "updatedAt": "bsonType": "date"
```

4.3. Оценка памяти полей коллекций NoSQL

- **Robots:** `_id` (12 B), `name` (~128 B), `scanRadius` (8 B), `weight` (4 B), `groupId` (12 B), `groupName` (~128 B), `model` (~128 B), `comments` (~512 B), `createdAt` (8 B), `updatedAt` (8 B). Итого один документ Robot ≈ 920 B.

- **Groups:** _id (12 B), name (~128 B), description (~512 B), status (~32 B), createdAt (8 B), updatedAt (8 B). Итого один документ Group ≈ 700 B.
- **Events:** _id (12 B), robotId (12 B), taskId (12 B), type (~32 B), message (~256 B), position (16 B), prevPosition (16 B), target (16 B), speed (8 B), prevTimestamp (8 B), metric (48 B), gridFsFileId (12 B), description (~256 B), timestamp (8 B), createdAt (8 B). Итого один документ Event $\approx 380\text{--}480$ B (в зависимости от состава опциональных полей).
- **Tasks:** _id (12 B), name (~128 B), groupId (12 B), groupName (~128 B), executionRobots (массив из 10 элементов по ~ 60 B = 600 B), type (~32 B), taskDetails (~512 B), taskStatus (~32 B), plannedRoute (массив из 250 точек по 8 B = 2000 B), поля дат (32 B). Итого один документ Task ≈ 3360 B.
- **Obstacles:** _id (12 B), points (массив из 20 вершин по 8 B = 160 B), bounding box (minX, maxX, minY, maxY = 16 B), name (~128 B), active (1 B), даты (16 B). Итого один документ Obstacle ≈ 365 B.
- **GridFS (Фото):** Метаданные fs.files (~128 B), бинарные блоки fs.chunks по 255 KB. Итого файл изображения $\approx 1,048,576$ B (1 MB).

5. РЕЛЯЦИОННАЯ МОДЕЛЬ ДАННЫХ (SQL)

Реляционная структура нормализована до 3НФ. Все связи «один-ко-многим» и «многие-ко-многим» вынесены в отдельные связующие таблицы с поддержкой ссылочной целостности (FOREIGN KEY).

5.1. Графическое представление модели

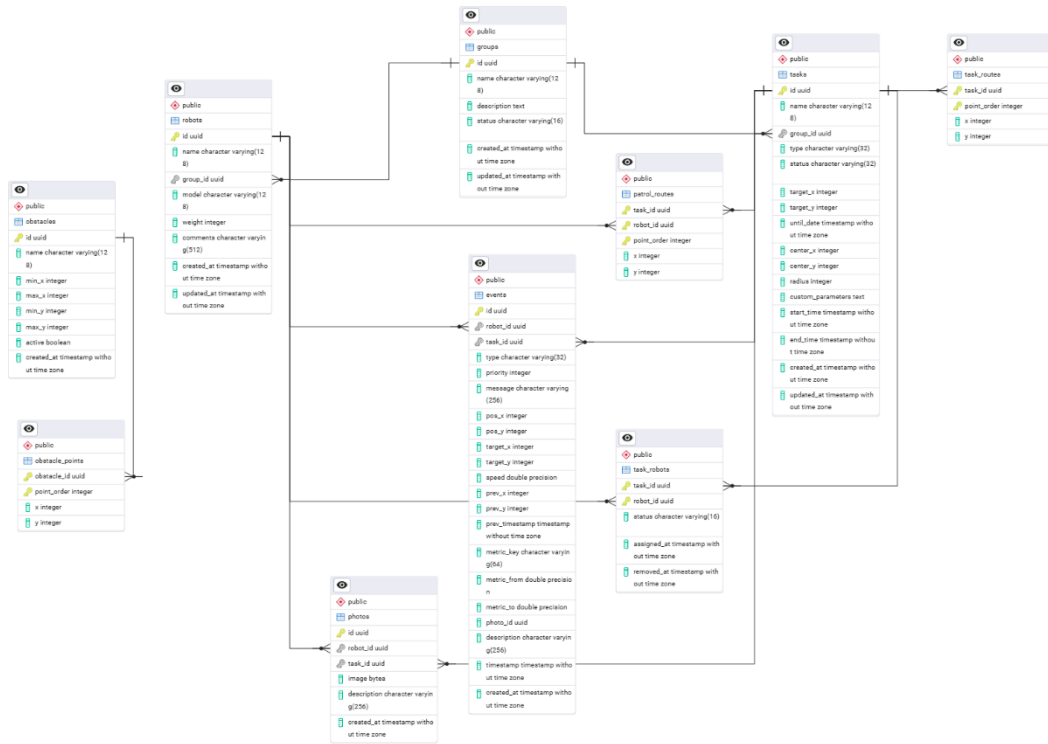


Рис.12 - Графическое представление модели

5.2. Структура таблиц

```
CREATE TABLE groups (  
    id UUID PRIMARY KEY,  
    name VARCHAR(128) NOT NULL,  
    description TEXT,  
    status VARCHAR(16) NOT NULL,  
    created_at TIMESTAMP NOT NULL,  
    updated_at TIMESTAMP NOT NULL  
);
```

```
CREATE TABLE robots (
  id UUID PRIMARY KEY,
```

```

name VARCHAR(128) NOT NULL,
group_id UUID REFERENCES groups(id) ON DELETE RESTRICT,
model VARCHAR(128) NOT NULL,
weight INT,
comments VARCHAR(512),
created_at TIMESTAMP NOT NULL,
updated_at TIMESTAMP NOT NULL
);

```

```

CREATE TABLE tasks (
  id UUID PRIMARY KEY,
  name VARCHAR(128),
  group_id UUID REFERENCES groups(id),
  type VARCHAR(32) NOT NULL,
  status VARCHAR(32) NOT NULL,
  target_x INT,
  target_y INT,
  until_date TIMESTAMP,
  center_x INT,
  center_y INT,
  radius INT,
  custom_parameters TEXT,
  start_time TIMESTAMP,
  end_time TIMESTAMP,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL
);

```

```

CREATE TABLE task_robots (
  task_id UUID REFERENCES tasks(id) ON DELETE CASCADE,
  robot_id UUID REFERENCES robots(id) ON DELETE RESTRICT,
  status VARCHAR(16) NOT NULL,
  assigned_at TIMESTAMP NOT NULL,
  removed_at TIMESTAMP,
  PRIMARY KEY (task_id, robot_id)
);

```

```

CREATE TABLE task_routes (
  task_id UUID REFERENCES tasks(id) ON DELETE CASCADE,

```

```
    point_order INT,  
    x INT NOT NULL,  
    y INT NOT NULL,  
    PRIMARY KEY (task_id, point_order)  
);
```

```
CREATE TABLE patrol_routes (  
    task_id UUID REFERENCES tasks(id) ON DELETE CASCADE,  
    robot_id UUID REFERENCES robots(id) ON DELETE CASCADE,  
    point_order INT,  
    x INT NOT NULL,  
    y INT NOT NULL,  
    PRIMARY KEY (task_id, robot_id, point_order)  
);
```

```
CREATE TABLE obstacles (  
    id UUID PRIMARY KEY,  
    name VARCHAR(128) NOT NULL,  
    min_x INT NOT NULL,  
    max_x INT NOT NULL,  
    min_y INT NOT NULL,  
    max_y INT NOT NULL,  
    active BOOLEAN NOT NULL,  
    created_at TIMESTAMP NOT NULL  
);
```

```
CREATE TABLE obstacle_points (  
    obstacle_id UUID REFERENCES obstacles(id) ON DELETE CASCADE,  
    point_order INT,  
    x INT NOT NULL,  
    y INT NOT NULL,  
    PRIMARY KEY (obstacle_id, point_order)  
);
```

```
CREATE TABLE photos (  
    id UUID PRIMARY KEY,  
    robot_id UUID REFERENCES robots(id),  
    task_id UUID REFERENCES tasks(id),  
    image BYTEA NOT NULL,
```

```

description VARCHAR(256),
created_at TIMESTAMP NOT NULL
);

CREATE TABLE events (
  id UUID PRIMARY KEY,
  robot_id UUID REFERENCES robots(id) ON DELETE CASCADE,
  task_id UUID REFERENCES tasks(id),
  type VARCHAR(32) NOT NULL,
  message VARCHAR(256),
  pos_x INT,
  pos_y INT,
  target_x INT,
  target_y INT,
  speed DOUBLE PRECISION,
  prev_x INT,
  prev_y INT,
  prev_timestamp TIMESTAMP,
  metric_key VARCHAR(64),
  metric_from DOUBLE PRECISION,
  metric_to DOUBLE PRECISION,
  photo_id UUID REFERENCES photos(id),
  description VARCHAR(256),
  timestamp TIMESTAMP NOT NULL,
  created_at TIMESTAMP NOT NULL
);

```

5.3. Оценка памяти полей таблиц SQL

- groups: ≈ 688 B
- robots: ≈ 824 B
- events: $\approx 436\text{--}496$ B
- tasks: ≈ 800 B
- task_robots: 64 B
- task_routes: 28 B
- patrol_routes: 44 B
- obstacles: 169 B

- obstacle_points: 28 B
- photos: $\approx 1,048,888$ B

6. СРАВНЕНИЕ МОДЕЛЕЙ

6.1. Удельный объем информации и направление роста моделей

Для проведения строгого анализа введем базовую независимую переменную:

- T — суммарное количество сгенерированных track-точек движения роботов.

На основании эмпирических допущений функционирования флота RMMS установим системные корреляции между сущностями:

- 1 миссия (M) генерирует 250 track-точек $\rightarrow M = T/25$.
- 1 миссия выполняется 1 логической группой (G) $\rightarrow G = T/250$.
- В 1 группе задействовано 10 роботов (R) $\rightarrow R = 10 \cdot G = T/25$.
- 1 миссия генерирует 50 системных событий логов (E) $\rightarrow E = 50 \cdot M = T/5$.
- 1 миссия производит 10 фотоснимков (F или P) $\rightarrow F = P = 10 \cdot M = T/25$.
- Количество препятствий (O) стационарно и равно 100 (по 20 вершин на полигон $\rightarrow OP = 2000$).

Математический расчет общего объема реляционной модели (S_{sql}):

Подставляя размеры строк таблиц и зависимости от T в интегральное уравнение:

$$S_{sql}(T) = 860R + 688G + 80T + 356E + 800M + 64TR + 28TP + 44PR + 169O + 28OP + 320L + 1048888P$$

После упрощения коэффициентов получаем итоговую зависимость объема:

$$S_{sql}(T) = 42192.192 \cdot T + 72900$$

- Доля графических бинарных данных (Фото): $41955.52 \cdot T$
- Доля текстово-числовых метаданных: $236.672 \cdot T$

Математический расчет общего объема нереляционной модели (S_{nosql}):

Учитывая вложенные массивы документов MongoDB (точки траектории внутри документа Tasks, роботы внутри задач):

$$S_{\text{nosql}}(T) = 1120R + 700G + 365O + 68T + 336E + 3360M + 349F + 1048576F$$

После упрощения и приведения подобных членов:

$$S_{\text{nosql}}(T) = 42153.24 * T + 36500$$

- Доля графических бинарных данных (Фото): $41943.04 * T$
- Доля текстово-числовых метаданных: $210.24 * T$

Направление роста: В обеих моделях ключевым драйвером роста является сущность Задачи (Tasks). Однако в реляционной модели создание одной задачи приводит к каскадному росту записей в 5 различных связанных таблицах (task_robots, task_routes, patrol_routes и др.), расширяя БД «вширь». В нереляционной модели рост носит строго линейный характер, увеличивая документы в рамках существующих коллекций за счет атомарного добавления логов в Events. В «нефотографической» части реляционная модель требует на $26.432T$ байт больше дискового пространства из-за накладных расходов на хранение явных индексов внешних ключей.

6.2. Анализ избыточности моделей

Под избыточностью понимается отношение фактического объема хранения к «чистому» (полезному) объему данных, очищенному от технических идентификаторов связей (UUID, ObjectId, внешние ключи).

Избыточность в SQL-модели:

Удаляемые технические поля (суррогатные ключи id, references):

- robots (32 B), groups (16 B), events (48 B), tasks (32 B), task_robots (32 B), task_routes (16 B), patrol_routes (32 B), obstacles (16 B), obstacle_points (16 B), photos (48 B).

Суммарный избыточный технический объем:

$$R_{\text{extra_sql}}(T) = 31.552 * T + 33600$$

Итоговая избыточность текстовой структуры (без учета фото):

$$R_{\text{sql}}(T) = (236.672 * T) / (205.12 * T) = 1.15$$

Избыточность в NoSQL-модели:

Удаляемые дублирующиеся поля денормализации (например, копия groupName внутри каждого робота и задачи для ускорения поиска):

- groupName в Robots (128R), position и taskId в Events (20E), массивы Robots в Tasks (2528M).

Суммарный избыточный объем денормализации:

$$R_extra_nosql(T) = 19.232 * T + 1600$$

Итоговая избыточность текстовой структуры NoSQL (без учета фото):

$$R_nosql(T) = (210.24 * T) / (191.008 * T) = 1.10$$

Вывод по избыточности: Накладные расходы реляционной модели на поддержание ссылочной целостности через суррогатные ключи UUID (16 байт) превышают накладные расходы NoSQL на контролируемое дублирование строк при денормализации.

6.3. Сравнительная таблица сложности запросов

№	Сценарий использования (Юзкейс)	NoSQL (MongoDB)	SQL (Таблицы)	Кто быстрее и почему? (Комментарий аналитика)
1	UC-01: Показать список роботов конкретной группы и отфильтровать их по весу.	Обычный поиск по одной коллекции robots.	Обычный поиск по одной таблице robots.	Ничья. Обе СУБД справляются мгновенно, если настроены правильные индексы по группе и весу.
2	UC-02: Открыть карточку робота и показать его последнюю позицию на карте.	Поиск в robots + выборка последней точки из	Поиск в robots + выборка последней точки из	Ничья. В обеих базах это два быстрых точечных запроса.

№	Сценарий использования (Юзкейс)	NoSQL (MongoDB)	SQL (Таблицы)	Кто быстрее и почему? (Комментарий аналитика)
		events.	events.	
		Сложность: $O(\log N)$	Сложность: $O(\log N)$	
3	UC-04: Создать новую задачу (миссию) и привязать к ней 10 роботов.	Запись одного документа в коллекцию tasks (список роботов вкладывается внутрь).	Запись в таблицу tasks + 10 записей в связующую таблицу task_robots.	NoSQL выигрывает. В MongoDB мы делаем всего одну операцию записи, а в SQL приходится плодить строки в разных таблицах из-за требований нормализации.
		Сложность: $O(1)$	Сложность: $O(R)$	
4	UC-04: Посмотреть детали задачи и список всех роботов, которые её выполняют.	Чтение одного документа из коллекции tasks.	Склеивание через JOIN трех таблиц: tasks, task_robots и robots.	NoSQL безоговорочно выигрывает. MongoDB просто открывает один готовый файл, где всё уже написано. SQL тратит ресурсы процессора на склейку таблиц «на лету».
		Сложность: $O(1)$	Сложность: $O(\log N + R)$	
5	UC-05: Вывести на экран карту и показать актуальные координаты вообще всех живых роботов.	Сканирование коллекции последних событий	Сканирование таблицы events с фильтрацией по	Ничья. Нам в любом случае нужно поднять из базы актуальные координаты для

№	Сценарий использования (Юзкейс)	NoSQL (MongoDB)	SQL (Таблицы)	Кто быстрее и почему? (Комментарий аналитика)
		events.	времени.	каждого робота (R).
		Сложность: O(R)	Сложность: O(R)	
6	UC-06: Найти роботов, которые проезжали близко (в радиусе N метров) от зоны препятствия "А".	Находим координаты препятствия, затем ищем точки в events по диапазону координат.	Запрос со склейкой (JOIN) таблиц events и obstacles по условию BETWEEN.	NoSQL выигрывает. В реляционных базах такие геометрические склейки без специальных (пространственных) индексов заставляют базу перебирать миллионы строк.
		Сложность: O(log N)	Сложность: O(N * M)	
7	UC-06: Найти роботов-нарушителей, которые превысили скорость.	Проверка поля speed в коллекции логов events.	Проверка поля speed в таблице events.	Ничья. Обе базы просто перебирают лог событий (E) и вытаскивают строки, где скорость больше нормы.
		Сложность: O(E)	Сложность: O(E)	

Примеры реализации сложных запросов из таблицы:

Юзкейс №6 (NoSQL / MongoDB):

```
const obs = db.obstacles.findOne({ "name": "A" });
db.events.distinct("robotId", {
  "type": "track_point",
  "position.x": { gte: obs.minX - N, lte: obs.maxX + N },
```

```
"position.y": { gte: obs.minY - N, lte: obs.maxY + N }  
});
```

Юзкейс №6 (SQL):

```
SELECT DISTINCT e.robot_id FROM events e  
JOIN obstacles o ON o.name = 'A'  
WHERE e.type = 'track_point'  
AND e.pos_x BETWEEN o.min_x - :N AND o.max_x + :N  
AND e.pos_y BETWEEN o.min_y - :N AND o.max_y + :N;
```

РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

Было разработано приложение, реализующее сценарии использования. Содержит страницы Dashboard, Statistics Map Tasks Robots Groups Events Visual logs, а также страницы для отдельных сущностей – роботов, групп, событий, заданий. Также реализована симуляция поведения роботов на базовом уровне: при выполнении заданий роботы симулируют движение и отправляют своё состояние раз в несколько секунд.

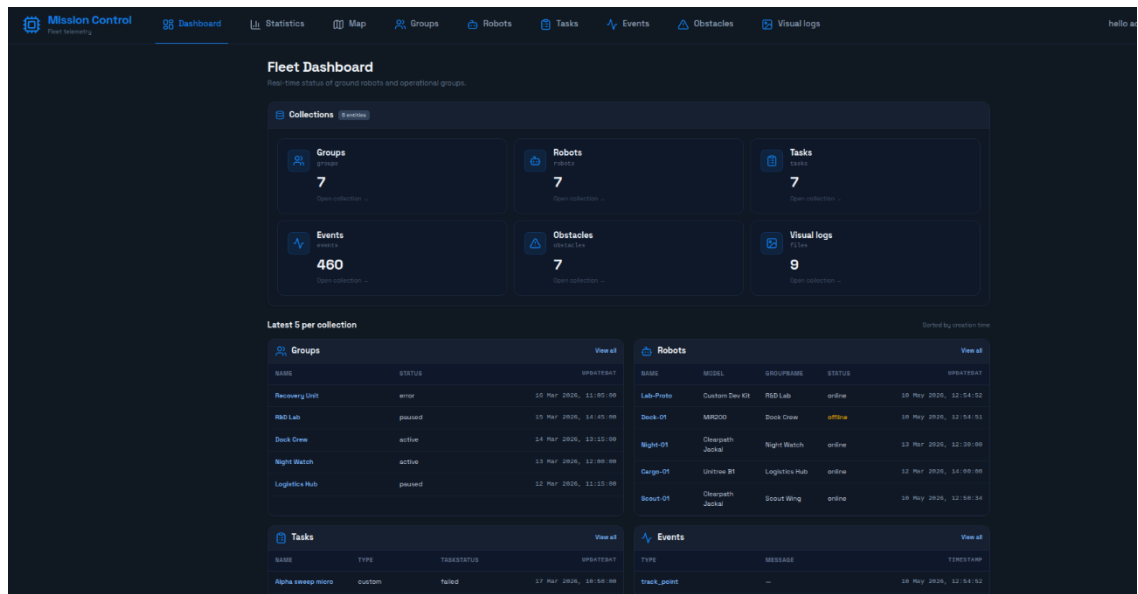


Рис.12 — Страница Dashboard

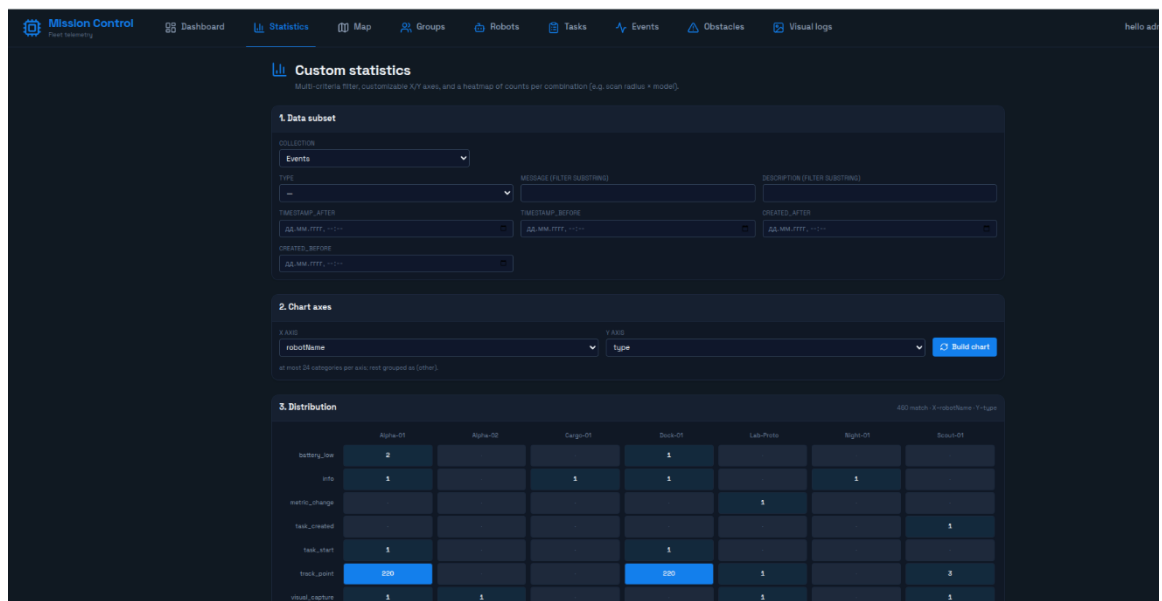


Рис.13 — Страница Statistics

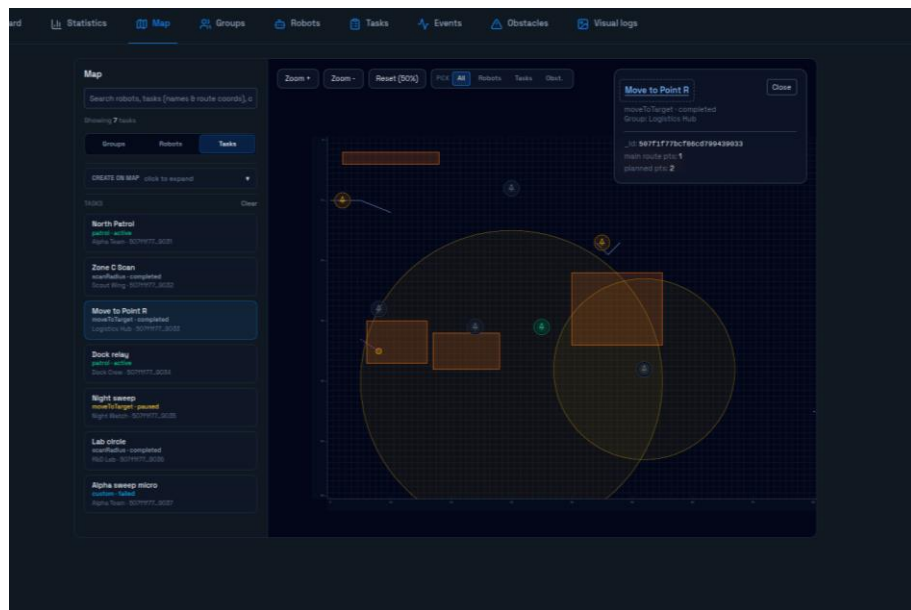


Рис.14 — Страница Map

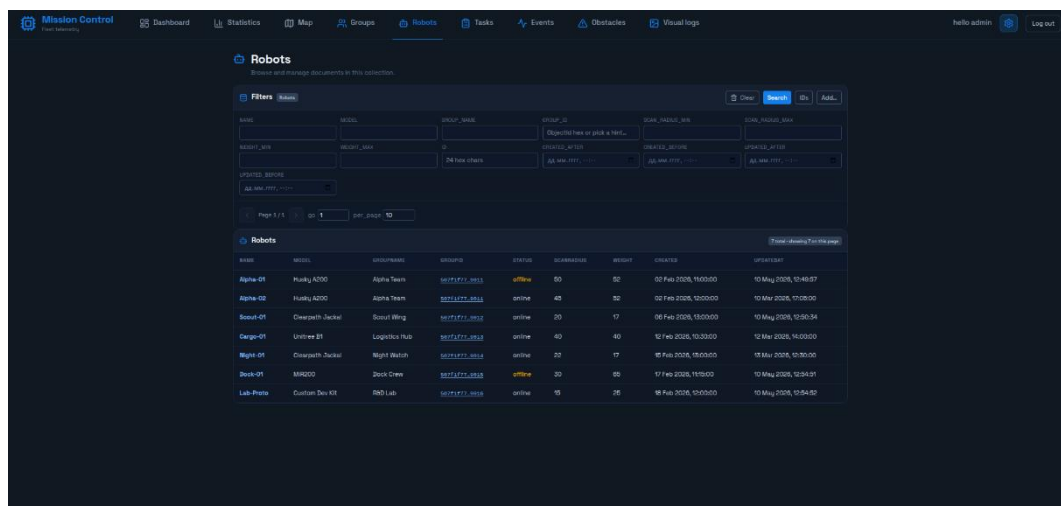


Рис.15 — Страница Robots

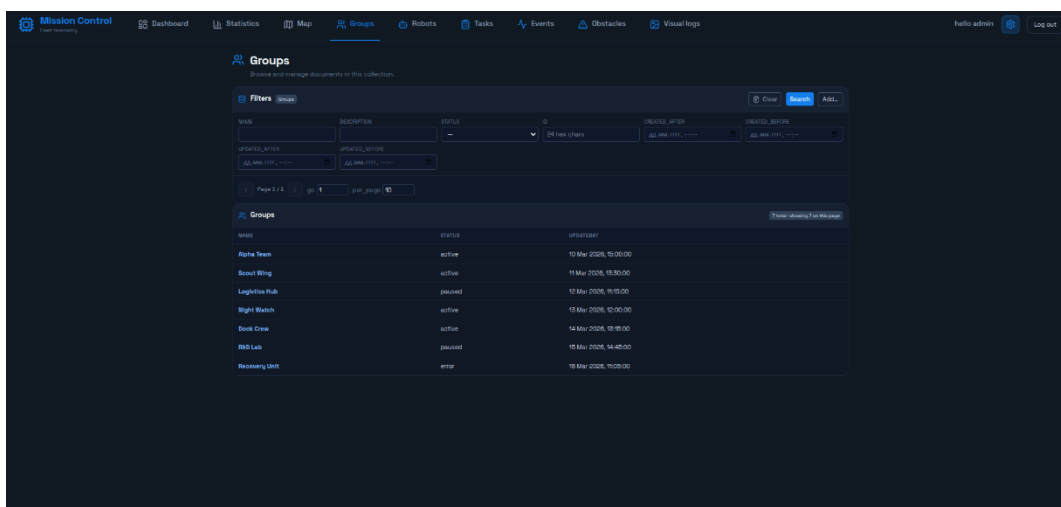


Рис.16 — Страница Groups

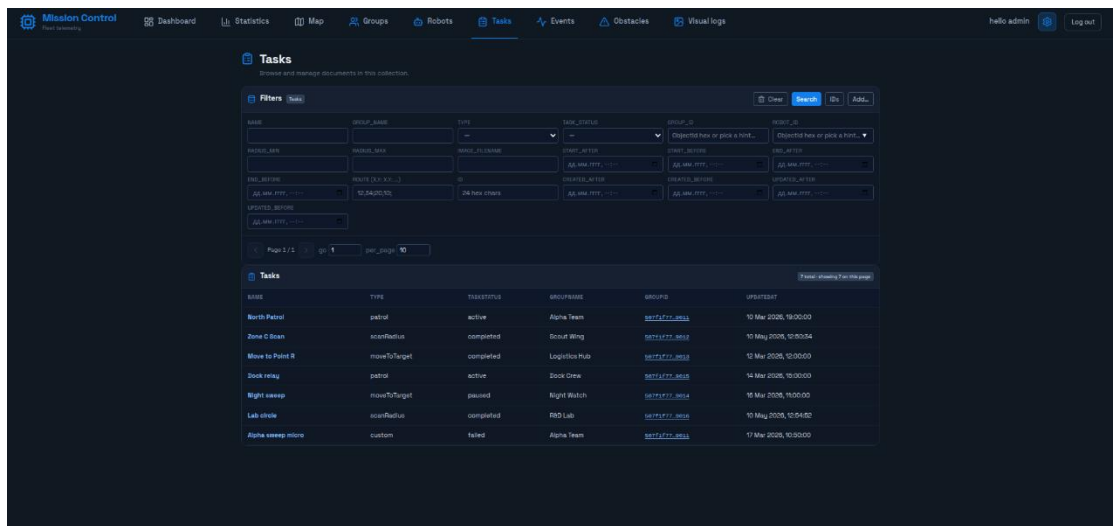


Рис.17 — Страница Tasks

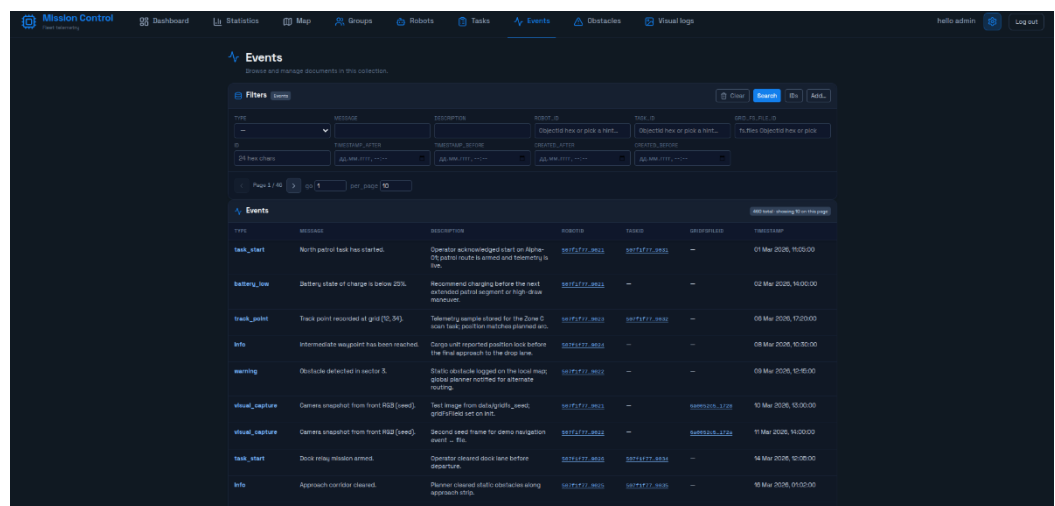


Рис.18 — Страница Events

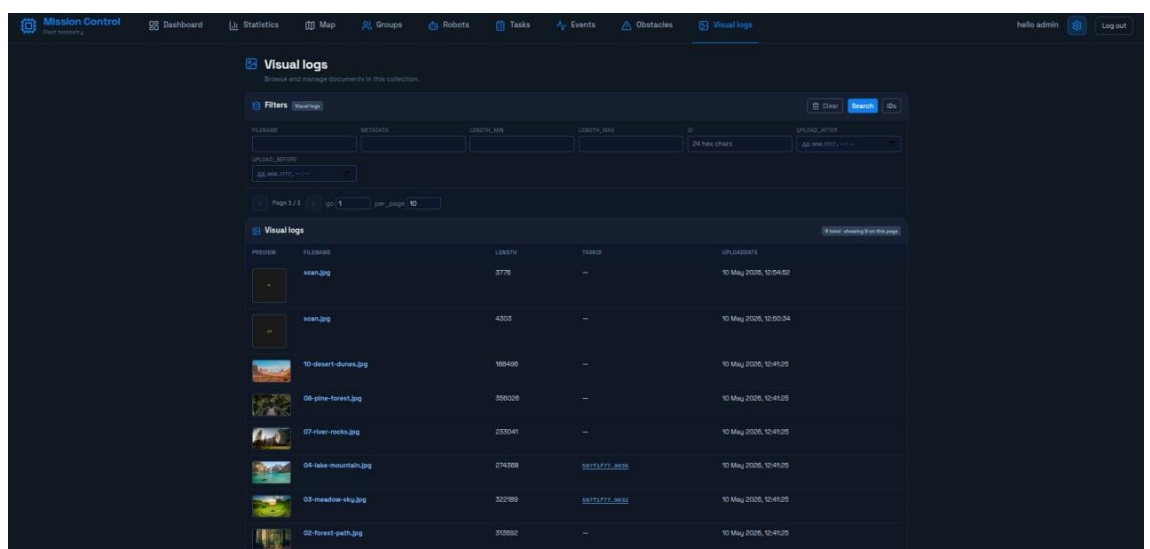


Рис.19 — Страница Visual logs

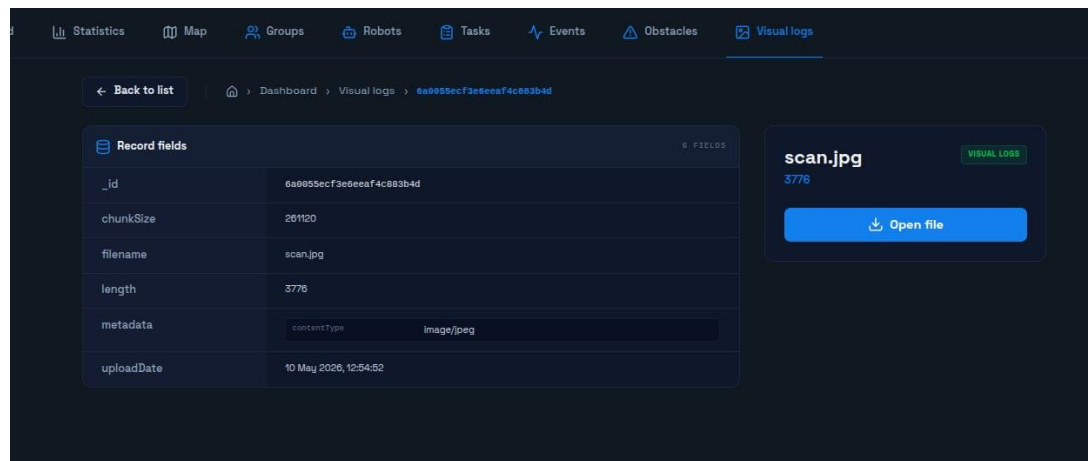


Рис.20 — Страница одного лога

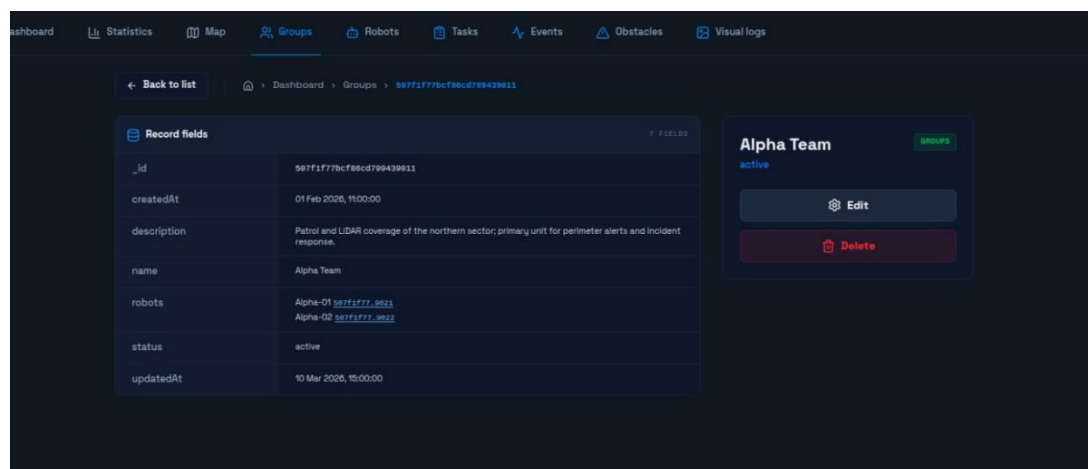


Рис.21 — Страница группы

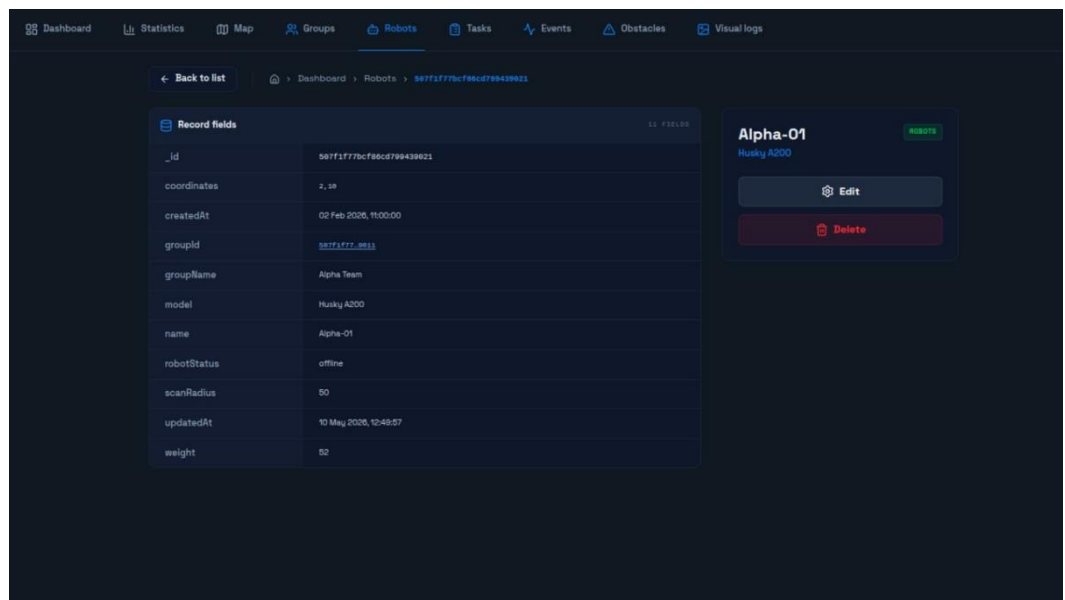


Рис.22 — Страница робота

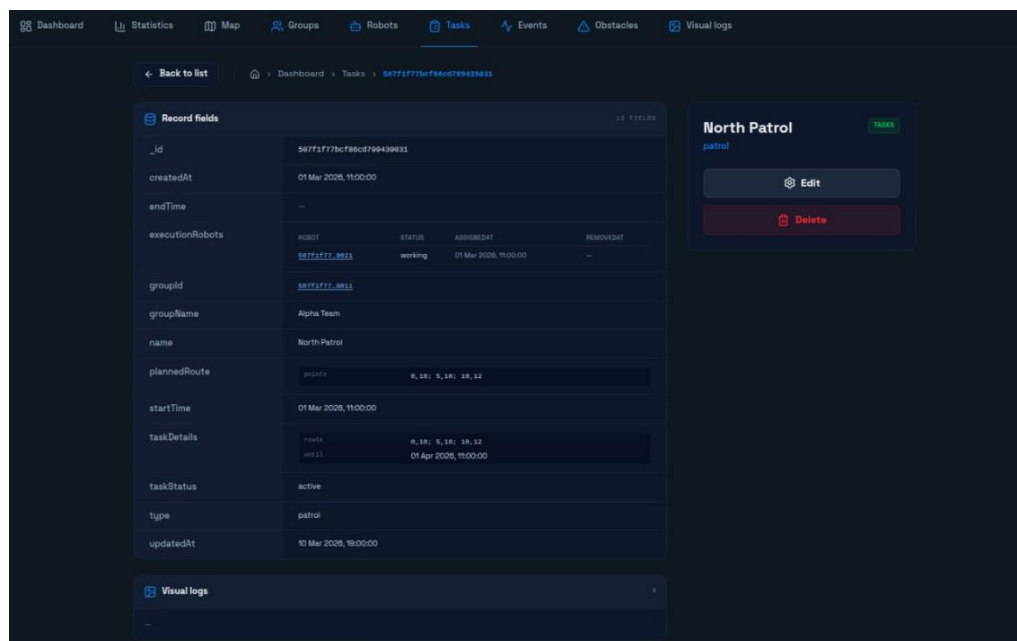


Рис.23 — Страница задания

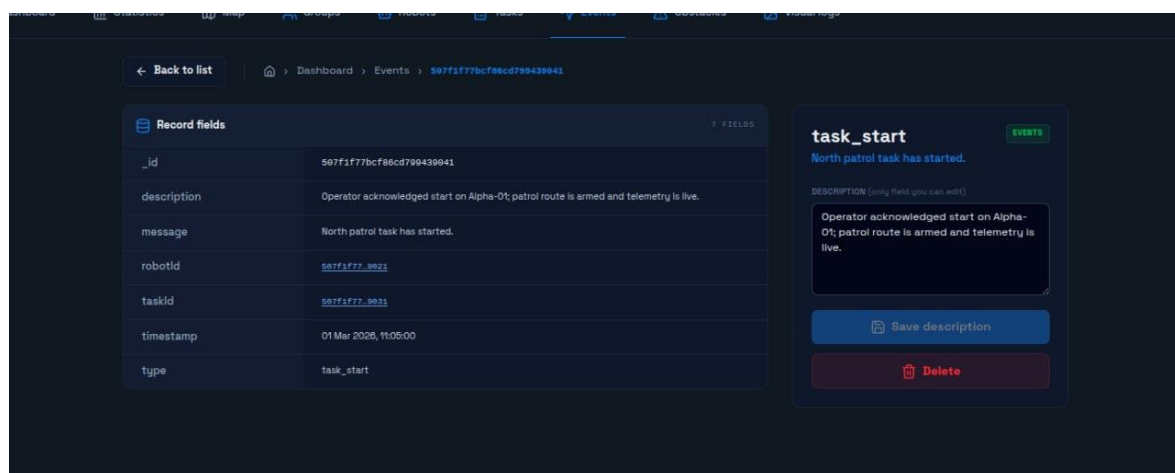


Рис.24 — Страница события

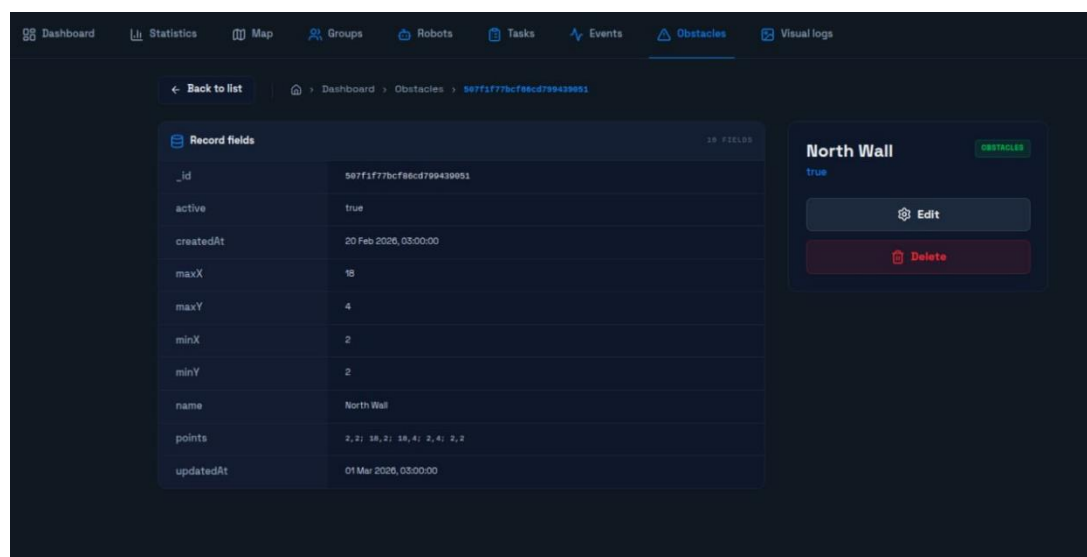


Рис.25 — Страница препятствия

Используемые технологии

- Backend: Python 3.12, FastApi[1] PyMongo.
- DB: MongoDB 7.0.15[2]
- Frontend: HTML, CSS, TypeScript

ЗАКЛЮЧЕНИЕ

В рамках выполнения данной работы спроектированы две полнофункциональные модели баз данных для системы управления флотом роботов RMMS. На основании математического моделирования и анализа вычислительной сложности операций сделаны следующие выводы:

1. Превосходство NoSQL структуры в задачах мониторинга: СУБД MongoDB показала более высокую эффективность при извлечении комплексных агрегатов. Использование встроенных документов позволяет получать полные данные за 1 дисковый запрос со сложностью $O(\log N)$, полностью ликвидируя необходимость в операциях JOIN, являющихся узким местом реляционных СУБД при масштабировании телеметрии.
2. Память и бинарные данные: Обе модели одинаково восприимчивы к тяжелым графическим данным (изображения с камер занимают до 99% общего объема хранилища). Однако встроенный протокол MongoDB GridFS позволяет автоматически партиционировать файлы на чанки в рамках одной СУБД.
3. Особенности обновления: Единственным недостатком NoSQL-модели является усложнение логики обновления данных при денормализации. Изменение имени робота требует выполнения каскадных обновлений во всех документах активных задач.

Итог: Для системы, характеризующейся высокой частотой чтения/записи временных рядов и графовых траекторий, документоориентированная NoSQL модель на базе MongoDB является оптимальным архитектурным решением

ПРИЛОЖЕНИЕ А

СБОРКА И РАЗВЕРТЫВАНИЕ ПРИЛОЖЕНИЯ

- Скачать репозиторий по ссылке[3]
- Запуск (из корня репозитория)
 - `cp .env.example .env`
 - `docker compose build --no-cache`
 - `docker compose up`
- После успешного запуска приложение будет доступно по адресу:
 - URL: `http://localhost:8000`
 - Логин: `admin`
 - Пароль: `admin`

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация MongoDB: <https://www.mongodb.com/docs/> (дата обращения: 24.05.2026).
2. Документация FastApi <https://fastapi.tiangolo.com> (дата обращения: 24.05.2026).
3. Репозиторий Github с исходным кодом: <https://github.com/moevm/nsql1h26-rob> (дата обращения: 24.05.2026).